

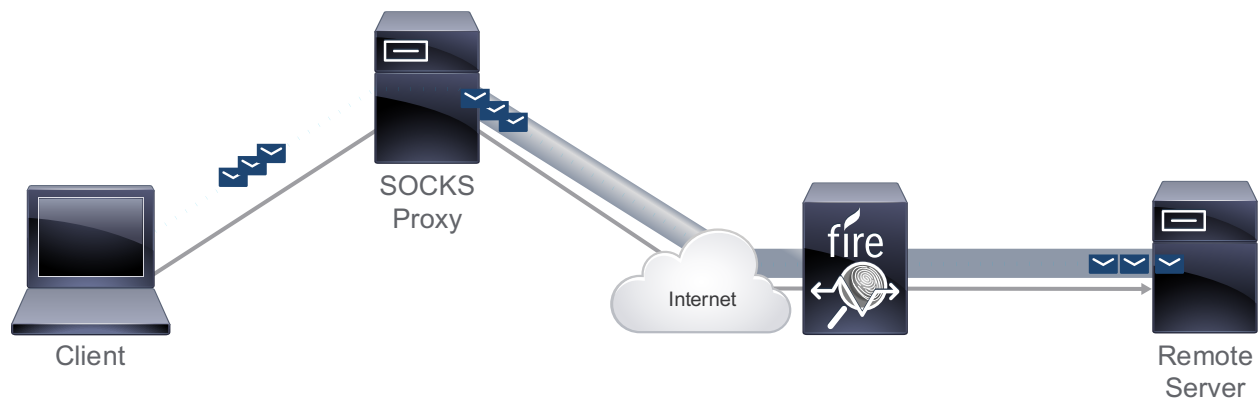
Technologies Used in Cobalt Strike and Metasploit

Overview

Proxying with the SOCKS Protocol

The Socket Secure (SOCKS) protocol is a network protocol that facilitates proxying. SOCKS proxies function as an intermediary between devices on the local network and internet resources, and they support any OSI Layer 5-and-above protocol. SOCKS proxies help ensure that internet activity through the proxy remains private, by masking the client's original private IP address with the proxy's own IP address. SOCKS is a protocol that is intended to act as a circuit level proxy for applications.

There are two versions of the protocol: SOCKS4 and SOCKS5. SOCKS5 is the latest version of the SOCKS protocol and is more secure.



A proxy server always creates a connection on behalf of the client to a remote server on the internet.

The transport protocol that is used to support the client to the SOCKS proxy server connection will depend on the SOCKS protocol version. Clients connect to SOCKS4 proxy servers by using TCP, typically on TCP port 1080. If the proxy server supports the SOCKS5 protocol, the client-to-proxy server connection can be either TCP or UDP.

SOCKS is a client/server-based model. A client's workstation must have a SOCKS proxy application installed, either in the application (such as PuTTY or Firefox) or deep in the TCP/IP stack where the client software will redirect packets into a SOCKS tunnel. The proxy server forwards packets from the client to the remote server and from the remote server back to the client. The SOCKS proxy server does not inspect the traffic. The SOCKS server creates a tunnel between itself and the remote server and then passes the client's traffic within the tunnel.

In the OSI model, SOCKS is a Layer 5 protocol (the session layer). It can proxy the traffic of any protocol above Layer 5, including common protocols, such as HTTPS, SMTP, FTP, POP3, and DNS. Because the SOCKS proxy supports many protocols, it could be considered a universal proxy, unlike

specific protocol proxies (application proxies), such as an HTTP or web proxy that supports only HTTP and HTTPS traffic.

SOCKS5 is an improved version of the SOCKS protocol, in that it provides more security features than SOCKS4. SOCKS5 creates a full TCP connection and supports authenticated and encrypted SSH connections between an SSH client and a remote SSH server. It's important to remember that SOCKS5 is simply the transport tunneling protocol. SOCKS5 transparently proxies all the traffic from the client through another network connection that is created by the proxy. SOCKS5 does not inherently provide the data encryption services. The application that is running on the client's system, such as an SSH client application (OpenSSH) or a web browser (supporting SSL/TLS), must provide the encryption services. All network traffic that is being directed into the client's SSH application can traverse the SOCKS5 proxy through a private SSH encrypted tunnel to a remote SSH server. The SOCKS5 protocol also adds support for the IPv6 protocol.

A SOCKS5 proxy can evade attempts by websites to block IP addresses by changing the client's original source IP address. The proxy can also be used to circumvent content providers' attempts to restrict access to services based on the geographical location of the client, by masking the client's original source IP address.

Named Pipes and the SMB Protocol

Metasploit and Cobalt Strike support a feature that allows communication sessions to pivot within an internal network by adding Server Message Block (SMB) Named Pipe listeners. An application running on a Windows system consists of one or more processes. Windows processes are often hidden services and programs running on the system. Processes are allowed to communicate with each other over named pipes, and the communication between two processes can be encapsulated within the SMB protocol. SMB connections have many legitimate purposes in a Windows environment. SMB traffic is common and is therefore allowed within a network. As an evasion technique, adversaries can disguise malicious content in SMB messages to avoid detection.

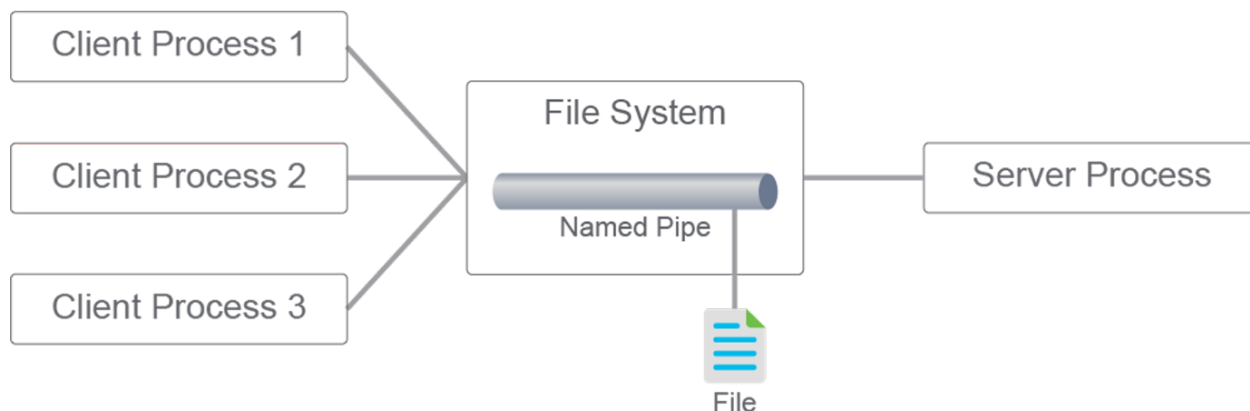
When a program is executed on a system, the CPU starts executing an associated process. At any moment, multiple processes might be executed by the CPU. These running processes can exchange data among themselves using OS components called named pipes.

If you use pipes, two separate processes that by design do not work together can communicate. One process writes to the pipe, and another process reads from it on a first-in, first-out (FIFO) basis. Commands are carried through the pipes between processes. The same principle is valid in plumbing piping to carry water, which inspired the name for this data exchange OS component. As plumbing pipes can combine to create pipelines to carry water, operating system pipes can combine to create processes for communication. Using multiple pipes, you can create a chain of processes that exchange data and create an OS pipeline.

An operating system provides two sorts of pipes:

- **Unnamed or traditional pipes:** A process creates an unnamed pipe when required, and the pipe lasts as long as the process exists.
- **Named pipes:** Depending on the operation system, named pipes can last as long as the system runs, even if the process that created it has ended. Named pipes appear as a file. Server processes are attached to it and can read and write instructions into the pipe.

Named pipes are located within the OS file system. In a Windows environment, named pipes use a dedicated file system called the named pipe file system (NPFS). The path to the root of the NPFS is [\\.\pipe\](#), and every new pipe has the location immediately under the root. For instance, the path to the "example" pipe would be [\\.\pipe\example](#).



A named pipe is a logical connection that forms between a client-side process and a server-side process. Processes that exchange data might be running on the same system or on systems that are reachable over the network. In Windows environments, the client and the server can reside anywhere within the Common Internet File System (CIFS). The CIFS shares files by using IP-based data communication. Clients and servers use a specific file system location, called a share, to exchange data. Clients and servers are pipe endpoints. Named pipes allow one server to have multiple clients.

You can share file information across the network by using the Server Message Block (SMB) protocol. CIFS data exchange is implemented with the use of SMB. The SMB protocol can use named pipes to exchange information. The clients access named pipes by using the SMB protocol messages. The named pipes that clients can access across the network are part of the shared location (pipe's share) named IPC\$. Within this shared space, only operations that are specific to named pipes are allowed.

Named pipes can be used to send and receive commands by using Cobalt Strike's SMB Beacon feature. As an evasion technique, adversaries can disguise beaconing command-and-control communication within SMB messages to avoid detection. The following two articles detail the advanced tactic of chaining named SMB pipes together within the Cobalt Strike workflow to embed beaconing C2 traffic within SMB named pipe communication:

<https://www.cobaltstrike.com/blog/named-pipe-pivoting/>

<https://www.cobaltstrike.com/blog/cobalt-strike-3-0-advanced-threat-tactics/>

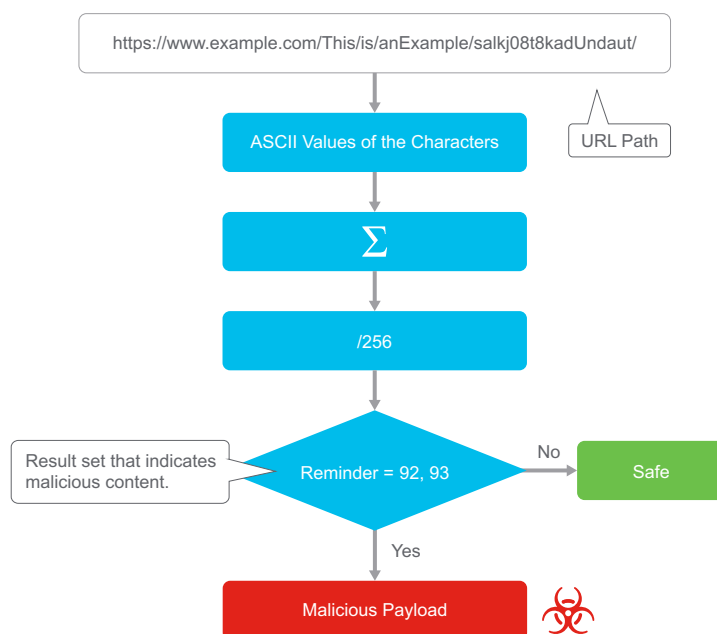
Raphael Mudge's post, "[Learn Pipe Fitting for all of your Offense Projects](#)", has some helpful additional details on the named pipes feature.

<https://www.cobaltstrike.com/blog/learn-pipe-fitting-for-all-of-your-offense-projects/>

Fingerprinting with Checksum8 and JA3 Fingerprints

Many attack frameworks, such as Metasploit and Cobalt Strike, create their URL paths to the servers that host malicious payloads. These URL paths include random characters, so that the new URL path always differs from the previously generated URL paths. Whatever these random characters are, when you perform a specific mathematical operation (referred to as a hash function) on the URL path, the result belongs to a set of fixed-sized character string values. These results are the fingerprint of the URL path. This method of verification, called fingerprinting, is a technique that is used to identify TLS encryption negotiations between a client and server. The distinct resulting pattern (fingerprint values) makes it possible to identify malicious URLs.

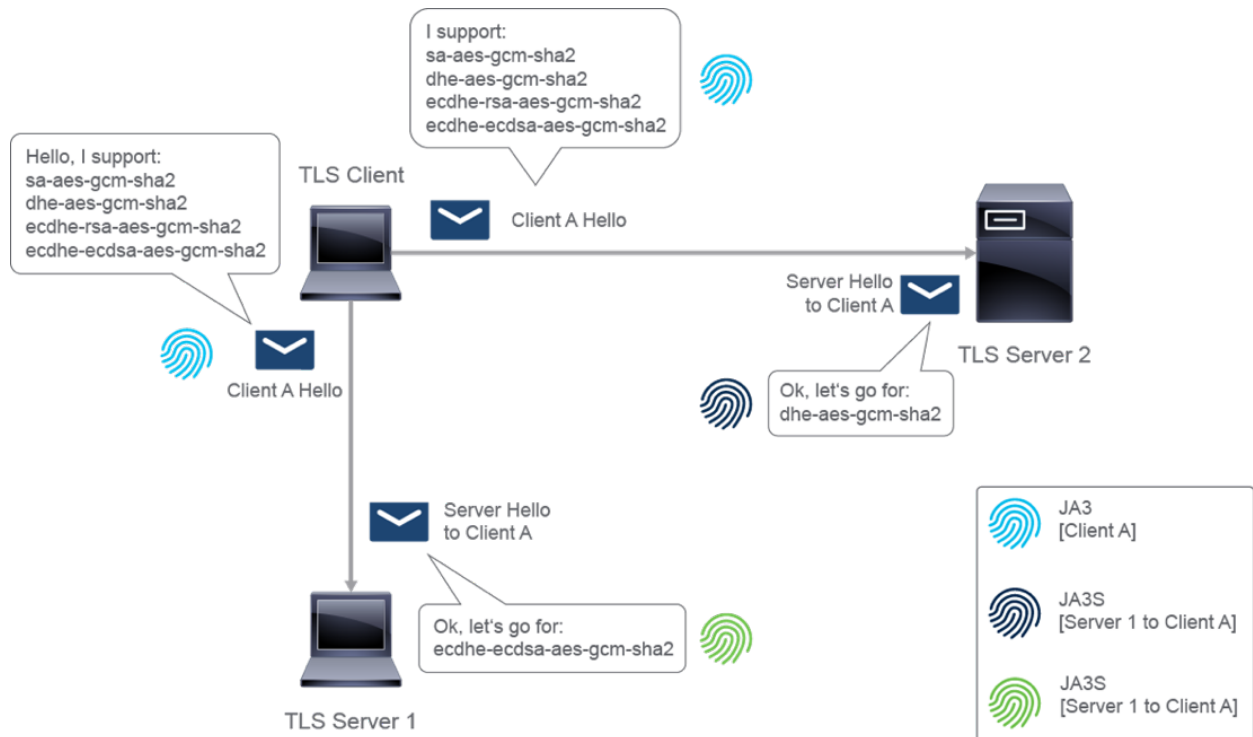
Checksum 8 Fingerprinting



The figure illustrates the *checksum8* fingerprinting method. It first sums the ASCII values of all the characters of the URL path (without the leading "/"). It then divides the sum by 256. If the remainder of the division is 92 or 93, the URL path represents a valid path to the malicious payload.

Another fingerprinting method is JA3 fingerprinting. JA3 creates an MD5 hashed 32-character string (a fingerprint) to identify the content, which can be a TLS protocol message. In Transport Layer Security (TLS) connections, JA3 is commonly used to identify the client, or the server, based on the TLS negotiation messages.

TLS is an authentication and encryption protocol that is used to secure internet communication. A TLS connection is initiated by a TLS handshake—a negotiation process in which the client and server agree on the set of parameters for the connection. This agreement includes the supported security algorithms known as the TLS cipher suite. The TLS negotiation uses TLS Hello messages, and both the client and the server exchange hello messages. They use these messages to inform each other of their supported encryption capabilities and to agree on a set supported by both parties.



Clients send their information in the TLS Client Hello messages. The content of the client hello depends on the cipher suite that the client application supports. Client applications (and application versions) can be designed so that they support a specific selection of algorithms, which makes them recognizable. The client application always offers the same set when they negotiate the connection. In other words, the client applications and application versions can be fingerprinted.

After the client proposes the cipher suite, the server creates a TLS Server Hello message in which it informs the client about the selected cipher suite. The server makes the selection based on the content that the client provided and on its own configuration.

The server response might differ for each new client, but the server always creates the same response to the same client's hello message. This response generates the same JA3-based fingerprint signature. The server's fingerprint is known as JA3S (note the S at the end), and its consistency makes it possible to recognize the server within an exchange with a specific client.

JA3 Fingerprint Examples

Standard Tor client:

JA3 = e7d705a3286e19ea42f587b344ee6865 (Tor Client)

JA3S = a95ca7eab4d47d051a5cd4fb7b6005dc (Tor Server Response)

Trickbot malware:

JA3 = 6734f37431670b3ab4292b8f60f29984 (Trickbot)

JA3S = 623de9Gdb17d313345d7ea481e7443c (C2 Server Response)

Emotet malware:

JA3 = 4d7a28d6f2263ed61de88ca66eb0 11e3 (Emotet)

JA3S = 80b3a14bccc8598a1f3bbe83e71f735f (C2 Server Response)

The figure shows examples of JA3 and JA3S fingerprints for known malware clients.

When an adversary has created malware code and that code has initiated the TLS connection, you might be able to recognize the malicious client through its JA3 fingerprint. From a beaconing perspective, the manner in which an adversary's command-and-control (C2) server establishes a TLS connection with the infected host does not vary. TLS negotiations between the C2 server and the host are not encrypted and are sent in cleartext. This action makes it possible to identify characteristics of the TLS encrypted communication by fingerprinting the TLS connection. However, the malware code might initiate the TLS connection by using another service that is common in the environment, such as Windows Socket. In that case, the malware client has the same JA3 fingerprint as legitimate clients, making it hard to detect. If the malicious servers create specific responses for specific clients, though, you might be able to detect malicious activities by inspecting the combination of the JA3 and corresponding JA3S fingerprints.

Americas Headquarters
Cisco Systems, Inc.
San Jose, CA

Asia Pacific Headquarters
Cisco Systems (USA) Pte. Ltd.
Singapore

Europe Headquarters
Cisco Systems International BV Amsterdam,
The Netherlands

Cisco has more than 200 offices worldwide. Addresses, phone numbers, and fax numbers are listed on the Cisco Website at <https://www.cisco.com/go/offices>.

Cisco and the Cisco logo are trademarks or registered trademarks of Cisco and/or its affiliates in the U.S. and other countries. To view a list of Cisco trademarks, go to this URL: <https://www.cisco.com/go/trademarks>. Third-party trademarks mentioned are the property of their respective owners. The use of the word partner does not imply a partnership relationship between Cisco and any other company. (1110R)